

RI.UVT.RO Redesign

Implementation Plan

Headless WordPress + React
Direcția de Relații Internaționale
West University of Timișoara

Focus: Practical implementation roadmap for rebuilding the DRI website through structured content architecture, WordPress CMS governance, React frontend development, REST API integration, Git workflow, QA, and launch preparation.

Technology Stack: WordPress Headless CMS + REST API + React + Vite
+ TypeScript
Team Q-UW9

Contents

1	Purpose of This Plan	2
2	Implementation Strategy	2
3	Phase-by-Phase Plan	2
3.1	Phase 1: Audit, Discovery, and Content Mapping	3
3.2	Phase 2: UX Architecture and Design System	3
3.3	Phase 3: Development Environment and Backend Foundation	3
3.4	Phase 4: WordPress Content Architecture	4
3.5	Phase 5: React Frontend Development	4
3.6	Phase 6: Integration, QA, and Launch Readiness	5
4	Role-by-Role Execution Plan	5
4.1	Role 1: Project Manager + Frontend Lead	5
4.2	Role 2: UX Researcher + Journey Designer	5
4.3	Role 3: WordPress Architect	6
4.4	Role 4: UI Designer + React Stylist	6
4.5	Role 5: React Developer – Pages and Components	6
4.6	Role 6: WordPress Developer – API and Content Types	7
4.7	Role 7: Content Strategist	7
4.8	Role 8: QA + Performance + Accessibility	7
5	Team Workflow Rules	7
6	Concrete Deliverables	8
7	Suggested Work Sequence	8
8	Conclusion	9

1. Purpose of This Plan

This document converts the redesign proposal into a practical implementation roadmap for a **Headless WordPress + React** development environment.

The project is not only a visual redesign. It is a structured digital product rebuild focused on information architecture, guided user journeys, content governance, modern frontend development, and maintainable technical delivery.

The implementation follows a separated architecture:

- **WordPress** becomes the backend CMS.
- **REST API** delivers structured JSON data.
- **React + Vite** becomes the public website frontend.
- **Laragon** provides the recommended local WordPress environment.
- **GitHub workflow** keeps development organized through branches and pull requests.

2. Implementation Strategy

The team will build the project as a real software product, not as a collection of disconnected pages.

Core implementation principle:

WordPress manages content. React manages the public user experience. The REST API connects both layers.

In practice, the team must create:

- WordPress pages, posts, media, and custom post types;
- structured metadata through custom fields;
- React routes and reusable layouts;
- dynamic API-connected views;
- clear navigation and user journeys;
- multilingual-ready structure;
- responsive, accessible, and optimized frontend pages.

3. Phase-by-Phase Plan

3.1 Phase 1: Audit, Discovery, and Content Mapping

Before coding, the team must understand the existing platform.

Goal: identify what exists, what is broken, what should be preserved, and what should be rebuilt.

Main actions:

- Review the current `ri.uvt.ro` website page by page.
- Identify broken links, duplicated route trees, test pages, outdated pages, and weak hierarchy.
- Identify the main audiences: incoming students, outgoing students, staff, partners, and international applicants.
- Decide what becomes Pages, Posts, Custom Post Types, or downloadable resources.
- Define the future sitemap.

3.2 Phase 2: UX Architecture and Design System

The team creates the structural and visual blueprint.

Goal: replace scattered navigation with clear institutional journeys.

Main actions:

- Finalize sitemap and route structure.
- Define navigation flows for each user group.
- Create page templates: homepage, programme page, support page, news page, contact page.
- Establish typography, spacing, cards, notices, buttons, and CTA patterns.
- Produce wireframes and visual direction aligned with UVT identity.

3.3 Phase 3: Development Environment and Backend Foundation

The technical base is created.

Recommended local workflow:

Laragon → WordPress admin → REST API → React Vite frontend.

Main actions:

- Clone the repository.
- Place the project inside the Laragon web root if needed.

- Create the local WordPress database.
- Configure the local WordPress instance.
- Install frontend dependencies.
- Create the local frontend environment file.

```
cd frontend  
npm install
```

```
# frontend/.env.local  
VITE_WP_API_URL=http://ro.uvt.ri.test/wp-json
```

```
npm run dev
```

3.4 Phase 4: WordPress Content Architecture

The backend becomes structured and scalable.

Goal: turn content chaos into a maintainable CMS structure.

Main actions:

- Create Pages for stable institutional sections.
- Create Posts for news and announcements.
- Create Custom Post Types such as Programmes, Opportunities, Partnerships, Documents, and Staff Members.
- Add custom fields for deadlines, countries, contacts, PDFs, coordinators, and categories.
- Import, rewrite, or rebuild cleaned content.

3.5 Phase 5: React Frontend Development

The public-facing experience is built.

Goal: build the interface visitors actually use.

Main actions:

- Build the React routing system.
- Build header, footer, navigation shell, and page layouts.
- Build reusable components such as cards, accordions, filters, search blocks, notices, and CTA sections.
- Fetch data from the WordPress REST API.

- Render pages dynamically from CMS content.
- Implement responsive and accessible layouts.

3.6 Phase 6: Integration, QA, and Launch Readiness

Everything is stabilized.

Goal: make the platform reliable, usable, and ready for public deployment.

Main actions:

- Test WordPress and React integration.
- Fix broken API responses.
- Test mobile responsiveness.
- Check accessibility.
- Optimize images and frontend bundles.
- Remove accidental `noindex`, `nofollow` rules from production pages.
- Finalize documentation.

4. Role-by-Role Execution Plan

4.1 Role 1: Project Manager + Frontend Lead

Main objective: keep the project technically aligned and own frontend standards.

Actual implementation work:

- Define React architecture.
- Create folder structure.
- Lead reusable component system.
- Review frontend merges.
- Coordinate delivery priorities.

4.2 Role 2: UX Researcher + Journey Designer

Main objective: ensure users can actually use the platform easily.

Actual implementation work:

- Design flows for students, staff, and partners.
- Improve navigation clarity.
- Define menus, breadcrumbs, and quick links.
- Validate usability decisions.

4.3 Role 3: WordPress Architect

Main objective: turn content chaos into maintainable backend structure.

Actual implementation work:

- Create page hierarchy.
- Define Posts vs Pages vs Custom Post Types.
- Configure slugs and menus.
- Manage backend content governance.

4.4 Role 4: UI Designer + React Stylist

Main objective: create the visual identity and implement it.

Actual implementation work:

- Design cards, sections, hero areas, and content blocks.
- Implement styling system.
- Ensure visual consistency.
- Improve responsiveness.

4.5 Role 5: React Developer – Pages and Components

Main objective: build frontend interfaces.

Actual implementation work:

- Build page templates.
- Build reusable components.
- Connect API data to views.
- Implement responsive layouts.

4.6 Role 6: WordPress Developer – API and Content Types

Main objective: make WordPress useful for a headless setup.

Actual implementation work:

- Register custom post types.
- Add custom fields.
- Expose required data in the REST API.
- Configure media and content handling.

4.7 Role 7: Content Strategist

Main objective: clean and structure information.

Actual implementation work:

- Rewrite confusing text.
- Add headings and summaries.
- Populate WordPress content.
- Keep labels understandable.

4.8 Role 8: QA + Performance + Accessibility

Main objective: make the final product reliable.

Actual implementation work:

- Test devices and browsers.
- Detect bugs.
- Verify API-fed pages.
- Improve accessibility.
- Optimize loading speed.

5. Team Workflow Rules

- Nobody works only theoretically.
- Reusable systems are preferred over one-off hacks.

- Backend and frontend must stay synchronized.
- Content must respect the agreed structure.
- Bugs are fixed continuously.
- Shared Git discipline is mandatory.

Git rule: do not commit directly to `main`. Work on feature branches and open pull requests.

```
git checkout -b feature/your-feature-name
git add .
git commit -m "feat: describe your change"
git push origin feature/your-feature-name
```

6. Concrete Deliverables

By project end, the team should produce:

1. Full audit report.
2. Final sitemap.
3. Design system.
4. Structured WordPress backend.
5. Working React frontend.
6. API-connected dynamic pages.
7. Improved content.
8. Tested mobile-ready platform.
9. Final documentation.

7. Suggested Work Sequence

1. Audit current site.
2. Define user flows.
3. Finalize sitemap.
4. Set up WordPress and React environments.
5. Build backend models.
6. Build frontend UI.

7. Connect REST API.
8. Populate content.
9. Test and optimize.
10. Prepare final presentation.

8. Conclusion

We are not redesigning only visuals. We are rebuilding content architecture, user journeys, and delivery technology through WordPress + React.

This project should be approached like a real software product build:

- with ownership;
- with systems;
- with maintainability;
- with clear Git workflow;
- with modern engineering discipline.